

How Anthropic uses Claude Code: Agentic Software Engineering at Scale — Daisy Hollman (中英对照)

演讲者 / Speaker: Daisy Hollman (Anthropic, Claude Code 团队)

频道 / Channel: NDC Conferences

发布 / Published: 2026-08-11

时长 / Duration: 60:25

来源 / Source: <https://www.youtube.com/watch?v=shZgedW15vg>

说明 / Note: 英文为 YouTube 自动字幕原文；中文为人工翻译，按内容划分为 10 个章节，便于对照阅读。

第 1 章 开场与自我介绍

Opening & Background Daisy 的自我介绍、技术背景，以及本场演讲的议程概览。

[00:09] Hey, how's everyone doing today? Um, having a good conference so far. Yeah, I'm the last talk of the day, so I really have to bring the energy if everyone wants to stay awake. So, uh, thanks for like sticking around to come to my talk. I really appreciate it.

嘿，大家今天过得怎么样？嗯，会议到现在还顺吧。对，我是今天最后一个演讲，所以要是大家还想撑住不睡着，那我可得把场子热起来。呃，谢谢你们留下来听我讲，真心感激。

[00:24] Um, my name is Daisy. I work at Anthropic. Um, I work on this uh side project that turned into something big uh called Claude Code. Um, I don't know how who in here has used Cloud Code before. Holy crap, that's a lot of people. Okay.

嗯，我叫 Daisy，在 Anthropic 工作。嗯，我做的本来是个副业项目，后来做成了大家熟知的大项目——Claude Code。嗯，不知道在座用过 Claude Code 的有多少。天哪，居然这么多。好。

[00:45] Um, that's really exciting. But I will get to go through some of the early slides first. Um, uh, a little fast and we'll get to some of the the bigger later slides. That's that's really cool. Um yeah uh I got the amazing privilege of joining anthropic right before Cloud Code went out. It was actually a a internal product called Claude CLI um that I first contributed to and literally the day after I contributed my first PR the uh the um name was changed to cloud code which I think was a much better name for marketing than cloud CLI even though us silly engineers thought CLI was going to be a much better name.

嗯，真让人激动。不过前面几页幻灯片我还是先带大家过一遍，嗯，讲快点，把后面更重要的内容留出来。那真的太酷了。嗯，对了，我特别走运，在 Claude Code 发布前就加入了 Anthropic。我最早贡献的是个内部产品，叫 Claude CLI；我提交第一个 PR 的第二天，名字就改成了 Claude Code——我觉得从市场角度看，这比 Claude CLI 强多了，虽然我们这帮工程师当时还傻乎乎地觉得 CLI 才是更好的名字。

[01:30] Um uh in that time I got to lead the design and implementation of plugins and agent teams. uh a few other small features here and there but uh plugins especially have been my passion in terms of both in terms of building community and building customizability. I'm really interested in fullscale software engineering not just agentic programming and I think I hope you'll understand the difference between those two at the end but I I really want people to be able to do their jobs with this and not just build cute prototypes.

嗯，那段时间里，我主要负责设计和实现插件（plugin）和 agent teams，另外也零散做了些小功能。不过我最有热情的还是插件——不管是对社区建设，还是对可定制性。我真正感兴趣的其实是完整的软件工程，不只是智能体编程；希望大家听完能明白这两者的区别。我真心希望人们能用它来干正事，而不只是做些好玩的小原型。

[02:02] Um my background is in programming languages. Uh I was involved in the C++ standards committee for almost a decade. I was a chair for a couple years. Um I've been very deep in the trenches of uh difficult

programming. I used to work on uh parallel and distributed programming models for supercomputers.

嗯，我的本行是编程语言。我参与 C++ 标准委员会快十年，其中有几年还当了主席。我一直泡在那些最难啃的编程领域里，早先做的是面向超级计算机的并行与分布式编程模型。

[02:24] So trying to figure out ways to make it easier for people to write code that needed to scale up to an entire data center. Um, and now I'm working on cloud code for kind of the same reasons. I think it's uh I want non-technical people and semi-technical people to be able to build real production uh software and I think that the this talk is more about figuring out ways to enable that for people around you. Um software engineers more and more are going to be about software engineering more and more is going to be about enabling that. Um, so yeah, where we're headed here. Um, well, I have a section that is about getting on the same page about what agentic programming even is, but I guess everyone in here has used cloud code, so that is um, we'll see how that section goes. If I get a lot of nods, I'll go really fast and I'll spend more time on the the later stuff. Um, we're going to talk about customization of cloud code. Like why do you need to do it? um how the context window is the kind of the basic primitive that you are working with when you do this customization. Then we're going to dig into how plugins are created in cloud

所以我当时就在琢磨，怎么让人更容易写出能扩展到整个数据中心的代码。嗯，现在做 Claude Code，原因其实差不多。我希望非技术人员、半技术人员也能真正写出能上生产的软件；这场演讲，更多是关于怎么帮身边的人做到这点。软件工程师这个行当，会越来越像「让更多人也能做软件工程」。嗯，所以接下来。嗯，我原本安排了一个环节，打算先和大家对齐一下「什么是智能体编程」，不过看这阵容，在座应该都用过 Claude Code 了，那这个环节就……看情况吧。要是大家纷纷点头，我就讲快点，把时间留给后面的内容。嗯，我们会聊 Claude Code 的定制化——为什么要定制？做定制时，你打交道的最基本原语就是上下文窗口（context window）。然后我们再深入看看插件在 Claude Code 里是怎么搭起来的——

第 2 章 从聊天机器人到智能体编程

From Chatbot to Agentic Programming 厘清 chatbot 与 agent 的界限，工具调用如何催生编码智能体。

[03:40] code, what they're composed of, how do you build out this knowledge base in your monorepo or in your real large scale software engineering environment that um enables Claude to act as a software engineer. and I'll talk through a few of the ways that we actually run cloud code uh within Anthropic and and uh even just on the cloud code team and and kind of what's coming next.

……插件由哪些部分组成，你又怎样在单体仓库（monorepo）或真实的大规模软件工程环境里，搭起一套知识库，让 Claude 能像软件工程师那样干活。我还会讲几个我们自己在 Anthropic 内部、乃至 Claude Code 团队里实际运行 Claude Code 的方式，以及接下来要做什么。

[04:07] So yeah, let's let's start like a little bit basic. Um some of you may understand this, I promise you it's going to get more technical, but uh like what is the difference between a chatbot and an agent? Uh so when we first started to get LLMs working, we were building chat bots with them and it really the interaction looked something like this, right? It was the human turn and then there's the model says something and then you go back to the human turn and then the model says something and you go back to the human turn. In 2024, we came up with this thing called um uh tool calling um which allows the the model to basically just ask the computer to do something. I'll go through the specifics of what that looks like, but probably most of you have seen this happen. Um and at first it was just kind of like the model could go do something, could go get the output, and then could make a decision based on that and then go back to the human. And kind of we slowly started adding more and more tools like it could do a second tool call based on the first tool call results and then it could do a third one and at some point

好，那我们从最基础的说起。嗯，这些你们有些人可能已经懂了，我保证后面会越来越技术化，但先问个问题：聊天机器人和智能体（agent）到底差在哪？嗯，最早我们让大语言模型跑起来的时候，是用它们搭聊天机器人，交互大概是这样：人说话，模型回话，回到人，模型再说，又回到人。到了 2024 年，我们搞出了所谓的「工具调用」（tool calling），它让模型基本上能请计算机去干点什么。

具体长什么样我后面会讲，不过你们大多数人应该都见过。嗯，一开始模型只是去干点事、拿回结果、基于结果做个判断，然后再回到人。后来工具越来越多——它能根据第一次工具调用的结果发起第二次，再来第三次，到某个时刻——

[05:14] we crossed over a boundary where we decided we needed a new name for the thing and starting call started calling them agents. I think this is the easiest way like I don't like the debate of what's an agent um or the discussion.

……我们跨过了一条线，觉得得给这种东西起个新名字，于是开始叫它们「智能体」（agents）。我觉得这是最好懂的说法——我不太喜欢那种「到底什么才算智能体」的争论。

[05:27] It's a very buzzwordy thing, but this is basically what led to the term agent is that you went from no tool calls to one tool call to a few tool to tool calls to at some point it's got enough autonomy that we're going to call it an agent. Um and uh coding agents or agentic programming harnesses or whatever term you want to use for them um like cloud code, like codecs, like cursor um are basically agents where the tools we gave them are like normal programming tools. Uh shell commands, file edits, uh CI related tools, the ability to run code, the ability to compile code, right?

这确实是个被说烂的词，但「智能体」这个名字基本就是这么来的：从没有工具调用，到一次、几次，到某个时刻它有了足够的自主性，我们就叫它智能体。嗯，所谓编码智能体、智能体编程框架（harness），或者你爱叫什么都行——比如 Claude Code、Codex、Cursor——本质上都是智能体，只不过我们给它们的是普通编程工具：shell 命令、文件编辑、CI 相关工具、运行代码的能力、编译代码的能力，对吧？

[06:11] Um, I mean, all of those kind of fall into shell commands, but it's like we're basically giving the agent the ability to do things that a programmer can do. And I'm saying programmer, not software engineer, because we're going to talk about what the difference is there or the gap is there. But, um, tool calls are shockingly simple. When I first started getting onboarded into this whole AI world um, from C++ world, I was kind of shocked at how like primitive it all is. It's like literally you tell the model that it can write JSON and something will happen when it writes a certain JSON and then you give back the results of doing the something to the model and stick it in the context window right after that. And that's like that's the entire thing, right? This is how the model runs something in bash is it says that the tool it wants to use is bash.

嗯，说到底这些都属于 shell 命令，但我们等于是让智能体能做程序员能做的那些事。我这里特意说「程序员」而不是「软件工程师」，因为接下来我们要聊这两者的区别。不过，嗯，工具调用简单得吓人。我当年从 C++ 世界刚跨进整个 AI 领域时，有点被震住了——这一切居然这么原始。本质上就是告诉模型它可以写 JSON，写到某种特定 JSON 时就会触发一个动作；然后你把执行结果回给模型，紧接着塞进上下文窗口。整个机制就这么简单，对吧？模型在 bash 里跑东西，方式就是声明它想用的工具是 bash。

[07:05] Here's the command. Then the harness goes and runs this the the command and then puts the output in a tool result block, right? Um it's it's like super easy to get your mind around and and everything in agent agentic harnesses is built around this idea.

这就是命令。然后框架（harness）去执行它，把输出放进一个工具结果块里，对吧？嗯，这个思路很好懂，所有智能体框架（harness）都是围着它转的。

[07:25] Um [snorts] and the ability to like act on this output and then create another tool use that is based on it and then continue that loop is what an agent is and everything is built around that, right? Um the big workhorse that really makes uh coding agents work is the edit tool.

嗯（笑），能基于这个输出采取行动，再据此发起下一次工具调用，并一直循环下去——这就是智能体，一切都是围着它转的，对吧？嗯，真正让编码智能体跑起来的主力工具，是那个「编辑工具」（edit tool）。

[07:48] Um and the edit tool this is literally the the schema for the edit tool that is in cloud code. um is uh so cla by the way cloud code made all of my slides. So when I wanted this, I said, "Hey, can you put like a tool call from your system prompt into here? Um, it yeah, so um it has to give the old string, gives the file name, the old string, and the new string. And it's like a yeah, it's a it's a find and replace. Uh there's no cursor, no selection, no quoting. Old string has to match bite for bite. if there's multiples of a old string, it either has to say up front that I expect there to be multiples or it just fails. Um, these are like really really primitive tools and I think there's a lot of iterating to be done in that space that I'll talk about later. But I think um, yeah, just to give you some idea of where we're coming from. Um, one of my favorite things to do with this is since Claude made these slides, the tool call to make the previous slide looked like this. um where it is writing JSON that contains some JSON that references an old slide.

嗯，这个编辑工具——屏幕上就是 Claude Code 里编辑工具的真实 schema。哦对了，顺带说一句，我这些幻灯片全是 Claude Code 做的。所以当时我想要这一页，就说：「嘿，你能把系统提示词里的一个工具调用放这儿吗？」嗯，可以的。它要给出文件名、旧字符串和新字符串，说白了就是查找替换。没有光标，没有选区，也不能加引号。旧字符串必须逐字节匹配；要是同一个旧字符串出现多次，它要么提前声明「我预期会有多个匹配」，要么就直接报错。嗯，这些都是非常非常原始的工具，我觉得这个领域还有很大的迭代空间，后面会讲。先给大家一个概念。嗯，我最爱举的例子是：既然幻灯片是 Claude 做的，那生成上一页的那个工具调用大概长这样——它写一段 JSON，里面还嵌着引用某一页旧幻灯片的 JSON。

第 3 章 模型能力的跃迁与「时间范围」趋势

Model Capability Leaps & the Time-Horizon Trend 模型在编辑任务上表现惊人；METR「智能体摩尔定律」与 Mozilla 实测案例；引出假设。

[09:00] The models are shockingly good at this. Like not making a single token mistake in this entire thing is pretty shocking to me at least. It the the mental model that I have about how agents work and what they're good at and what they're bad at really still doesn't even jive with this. I'm like that is not something a human could do. character for character if they had to. So like this gives you some nuance into like where the agent what the agent is doing, what the agent might be good at, what it might not be good at. It had no trouble producing this. No errors, no multiple attempts. Probably the last time I saw a string replace error or an edit tool error was something like I don't know last February of 2025 or maybe April of 2025.

模型在这件事上强得惊人。整段内容一个 token 都没出错，至少让我很震惊。我脑子里那套「智能体怎么工作、擅长什么、不擅长什么」的心智模型，到现在都没完全跟上这个现实。我会想：这根本不是人类能做到的——要人逐字符地做，根本做不到。所以这也让你更细致地看清：智能体在做什么、可能擅长什么、可能不擅长什么。它生成这个毫无困难，没报错，也没反复试。我上一次看到字符串替换或编辑工具出错，大概还是 2025 年 2 月，或者也许是 4 月的事。

[09:53] The models have progressed a lot past that because this is an easy thing to train. Um yeah. So um and then you have these increasingly like because it can handle this complexity, right? You can you can make the tool calls increasingly complex. You can make it be able to do more and more complex things and that leads to these longer time horizon tasks being possible. And there's this this chart from um an AI research organization called Meter that some of you who's seen this before? Give me some idea. Okay, a few people. um as to how it's it's been called the the uh Moore's law of agentic I don't know the Moors law of agents or something is that um roughly speaking the time horizon for the task that models can do uh with a 50% success rate is doubling about every four months. Um you'll see that this starts this kind of stops at the beginning of this year.

模型在这方面早就远远进步了，因为这本来就很好训练。嗯，对。然后，因为模型能处理这种复杂度，你就可以把工具调用越做越复杂，让它干越来越复杂的事，于是那些时间跨度更长的任务才变得可行。有一张来自 AI 研究机构 METR 的图——有人见过吗？让我看看，哦，有几个人。嗯，它被叫做「智能体的摩尔定律」之类的。大致意思是：模型能以 50% 成功率完成的任务，时间跨度大约每四个月翻一倍。嗯，你们会看到这条曲线基本就停在今年年初附近了。

[10:58] We're now two model versions later, opus 47 and 48. And it's because the um the institute actually decided that there's kind of the error bars are too big as to what counts as a 16-hour task at this point. And so this this chart is almost no longer useful, but it's been a pretty steady trend um over the past uh three years. I don't know about putting a 2019 uh here, but 2022 through 2026. This gives you an idea of where we're headed and what the trend looks like. Um, and so like you can start by assuming that this is going to plateau at some point. It it will, but like I think that if you had back in the 80s or 70s bet on Mo's law uh plateauing, you'd be in a very different spot than if you hadn't, right? So I think it is worth considering the possibility at least that this trend continues at least for another year or two. Um and you know exponential trends are pretty crazy in terms of the way that they change how we work. Um and this is starting to show up in in hockey stick like charts. This is uh from April. Mozilla uh Foundation put out a chart of the um security vulnerabilities, security bug fixes that 现在又过了两个模型版本，是 Opus 47 和 48。这是因为该机构其实觉得，到了这个阶段，「16 小时任务」的边界误差已经太大了。所以这张图几乎没什么参考价值了，但在过去差不多三年里，趋势一直相当稳。我不知道要不要把 2019 标上去，不过 2022 到 2026 是这样。这让你大致看清我们往哪走、趋势什么样。嗯，你可以先把「它迟早会到平台期」当假设——它当然会——但要是你在七八十年代就押摩尔定律会见顶，你今天的处境会和没押注的人完全不同，对吧？所以我觉得，至少值得考虑这种趋势再持续一两年的可能性。嗯，指数级趋势在改变我们工作方式这件事上，相当疯狂。嗯，这已经开始体现在「曲棍球杆」状的图表里了。这是今年 4 月的事——Mozilla 基金会发了一张图，统计他们在——

[12:22] they shipped in the month of April with uh with access to our latest model. Um and it was literally more than they had had fixed in the past 15 months combined. And that's keep in mind, right? Like it's not like they didn't have AI tools in the past 15 months.

……在接入我们最新模型后，今年 4 月一个月内修复的数量。嗯，这个数字 literal 超过了他们过去 15 个月修复量的总和。但请记住，这可不是说他们过去 15 个月没有 AI 工具。

[12:41] This is like the difference between the most recent AI tools and the ones that they had in the past 15 months. Um, and yeah, um, this gives you some idea of the practical uses here, right? This is these are this is fixing real bugs in software. And I don't know about you, like fixing bugs in software is a use is useful work, I would say. So when somebody tells me, "Oh, I don't know that agents can do useful work." I like to point them to something like this.

这体现的是最新 AI 工具和他们过去 15 个月用的工具之间的差距。嗯，对，这让你对实际用途有个概念，对吧？这是在修软件里真实的 bug。我不知道你们怎么想，但我觉得修 bug 算是有用的工作。所以每当有人跟我说「哦，我不觉得智能体能做有用的工作」，我就喜欢拿这种例子指给他们看。

[13:11] It's like, "What if you never had to fix a bug in your software ever again, right?" Like, I don't know anyone who likes recreational debugging. Um, I like making things, right? And debugging is a thing that ended up having to do because I like making things.

我就想说：「要是你再也不用修软件里的 bug 了呢，对吧？」我不认识谁会把调试当消遣。嗯，我喜欢创造东西，对吧？而调试是因为我想创造，才不得不去做的事。

[13:29] So like for the rest of this talk, I'd like to consider the hypothetical of what does engineering look like if and when agents can write better code than you, right? Um like and keep in mind I am a person who used to give talks on like C++ metrogramming minutia, right? Like I was so proud of how good of a coder I was. Um, and the past year and a half at Anthropic has really convinced me that uh it doesn't matter. Um, and I'm not going to necessarily sit here and try and convince you that this hypothetical will become a reality. Totally valid opinion if you don't think that it's worth uh contemplating. But but if you do, like I'd like to build on that hypothesis. What does agentic programming look like? What does harness design look like? What does context engineering look like in a way that we can build this into something that can really manage monorreosized uh software projects? 所以在接下来的演讲里，我想和大家一起做个假设：当智能体能写出比你更好的代码时，工程学会变成什么样？嗯，别忘了，我可是曾经专门讲 C++ 模板元编程细枝末节的人，对吧？我当年对自己的编码水平可自豪了。嗯，但在 Anthropic 这一年半，真让我相信那已经不重要了。嗯，我并不一定要

坐在这儿劝你们相信这个假设会变成现实——你要是觉得不值得想，那也完全合理。但如果你愿意，我想在这个假设上继续推演：智能体编程长什么样？框架设计长什么样？上下文工程（contextengineering）又该怎么做，才能把它做成真正能管理单体仓库级软件项目的东西？

第 4 章 为什么要定制模型（三要素）

Why Customize? (Three Pillars) 访问权、机构知识、工具链——以及为何「上下文学习」本质是文本文件。

[14:33] Um, cloud code just happens to be the one I build in my day job. Copilot, cursor, codeex, all all of this information should transfer. Like I am not here I'm an engineer. I'm not here to give an advertisement. I'm here to give an academic talk about the discipline of agentic software engineering. Um, so if you if you hear me say quad a bunch of times, meaning like an LLM, I'm sorry. That's just because I'm used to saying that in my day job, but I don't actually mean that, right? I really am interested in the direction that software engineering is headed.

嗯，Claude Code 恰好是我日常工作里搭的那个。Copilot、Cursor、Codex，这些思路全都能迁移。我是个工程师，不是来打广告的——我是来做一场关于「智能体软件工程」这门学科的学术性演讲。嗯，所以要是你们听我时不时蹦出「quad」（指大语言模型），请见谅，那只是我工作习惯，并不是那个意思，对吧？我真正关心的是软件工程正在走向的方向。

[15:09] Um, okay. So let's talk about why do you even need to customize the model in the first place, right? If we are headed towards AGI, quote unquote, it's generally intelligent. Why do I need to tell it anything at all? Um, and there's there's three subpoints here. There's it needs to have access to the things it needs to access to do its job. It needs to know about all of the institutional knowledge that's available to you or to any other employee of your company. and it has to have the tooling to get all of that information into the context window correctly. So, we're going to talk through what that looks like. But the the fundamental thesis that I'd love for you to walk away with from here is that if Claude can't do everything you can do, it can't do your job with you, right? Um, out of the box, it really only sees like a repo and a shell. And like we don't even let it go out of the folder that you started in by default, right? And there are good security reasons for that, but like I don't know anyone who's a professional software engineer who's like I am going to spend my entire day without looking at any information anywhere that's not in this folder.

嗯，好。那我们来聊聊，为什么你首先得对模型做定制？如果我们在走向所谓的通用人工智能（AGI），它已经普遍智能了，为什么我还得告诉它任何事？嗯，这里有三点：它得能访问完成工作所需访问的东西；它得了解你和公司其他员工都能接触到的机构知识；它还得更有相应的工具，把这些信息正确送进上下文窗口。我们一个一个讲。但我最希望你们带走核心理念是：如果 Claude 做不到你能做的每件事，它就不可能和你一起完成你的工作，对吧？嗯，开箱即用，它其实只看得见一个仓库和一个 shell。而且默认情况下，我们甚至不让他走出你启动时的那个文件夹，对吧？这有充分的安全理由，但我不认识哪个专业软件工程师会说「我整天都不看这个文件夹之外的任何信息」。

[16:24] Um it's is kind of fine for zero to one projects. I think this is part of the reason why people see so much more success with like rapid prototypes and zero to one projects. Like part of it is that that is really easier, right? Like even if you're a regular software engineer, that's that's a human software engineer that that is an easier thing to do. But part of it is that like the constrained context, the the the boundaries we put it in uh negatively impact the agent even more. uh it's rarely enough to do high quality software engineering at really large scales and um I mean another thing about this that I don't think people think about enough is the fact that most professional software engineering doesn't live in the source code right source code is about programming software engineering is about much much more than just programming right um so like giving claude or giving your model access to the things you have access to in order to do your job will help it do your job or do your job with you better.

嗯，这对从零到一（0→1）的项目还凑合。我觉得这也是为什么人们在快速原型和 0→1 项目上看到更多成功——一方面这本来就更容易，对吧？就算是普通的人类软件工程师，这也相对容易。但另一方面，受限的上下文、我们给它划的边界，对智能体的负面影响更大。在真正大规模的尺度上，它很

少够用。而且我觉得有件事大家想得不够多：大多数专业软件工程并不存在于源代码里——源代码关乎「编程」，而软件工程远不止编程，对吧？所以让 Claude 或你的模型能访问你完成工作所能访问的那些东西，会帮它更好地完成工作，或与你协作得更好。

[17:30] Um like we are really not about trying to replace ourselves necessarily. We are about trying to multiply ourselves, right? And there's some subtlety there that in terms of like how scary the impact is on society that I won't go into, but like um for the sake of an academic talk, let's talk about like I think every one of us would like to have five of us at our company and would feel very good about the um productivity that you get out of that. So where does your work actually live? Um like where are most of the decisions made? Right? the decisions aren't actually made in the source code unless you have a very different uh software engineering environment than the one I'm used to at least, right? You you have a team chat, you have you have CI where you find out whether or not things work. You have dashboards that show you what's going on in production. Uh you have internal docs, design docs, things like that. the the tip that I give people and it's like actually a an experiment worth trying.

嗯，我们其实不一定要取代自己，我们是要「放大」自己，对吧？这里有些微妙之处，关于它对社会影响的吓人程度，我就不展开了；但就一场学术演讲而言，我想说：我们每个人都希望公司里有五个「自己」，并且会对由此带来的生产力感到高兴。那么，你的工作到底存在于哪里？嗯，大多数决策是在哪里做出的？对吧？除非你的工程环境和我习惯的很不一样，否则决策其实并不在源代码里，对吧？你有团队聊天群，有 CI 用来知道东西能不能跑通，有 dashboard 显示生产环境的状况，还有内部文档、设计文档等等。我给别人的建议——这其实是个值得一试的实验。

[18:34] Although like depending on how much time you have for this kind of thing, try doing a full full day of work without leaving the um without leaving the terminal, right? And if if you can't do your full job, then Claude can't do your job with you, right? Uh, if you get a Slack message and you can't respond to that Slack message by telling Claude what to say for you, then Claude can't answer that Slack message for you or Claude can't act on the information in that thread, right? Um, if you have a CI failure and you're like copy pasting the CI failure into the prompt, that that's something you're doing for it and it can't do for you anymore. Um, yeah. So uh the second aspect of this is is knowledge in general. Um and when I say knowledge I there is some of some aspects of this that are like oh this just wasn't in the training data. Um, but I think people tend to over access oops people tend to overindex on wasn't in the training data and and the reality is that there are things that are just like can't be trained into the model because different companies do them differently, right? Like your codebase's conventions are something that the model needs to

只要你时间允许，不妨试一整天不离开终端就把工作干完，对吧？如果你连完整的工作都做不了，那 Claude 也不可能和你协作完成你的工作，对吧？嗯，要是你收到一条 Slack 消息，却没法通过对 Claude 说「帮我回这条」来回应，那 Claude 就替你回不了，也没法利用那个会话里的信息，对吧？嗯，要是 CI 挂了，你还得手动把失败信息复制粘贴进提示词，那这就成了你替它做的事，它自己做不到。嗯，对。所以这第二点，是广义上的「知识」。嗯，说到知识，有些方面确实是「训练数据里没有」。但我发现人们容易过度强调「训练数据里没有」这一点；现实是，有些东西根本没法训练进模型，因为不同公司做法各不相同，对吧？比如你的代码库规范，是模型必须——

[19:57] know about but can't be trained into the model because different companies do different things different ways. your institutional memory, things that your team tried two quarters ago and never saw the light of day. Even if we had a perfectly trained model, we would need a way for the model to access this information. It's not even public, right? Um things that changed last week, right? Models have a training cutoff.

……了解、却没法训练进模型的，因为不同公司以不同方式做事。还有你的机构记忆——比如团队两个季度前试过、却没能落地的东西。即便我们有一个完美训练好的模型，我们仍需要一种方式让模型能访问这些信息，因为它们根本不是公开的，对吧？嗯，还有上周刚变的东西，对吧？模型都有训练截止日期。

[20:22] um things that are just yours, internal APIs, internal vocabulary, definitions for terms that your codebase uses and that your design docs use and how to find those definitions. Um so customization is knowledge

customization is closing this gap between what the model knows inherently and what your team knows.

嗯，还有那些只属于你自己的东西：内部 API、内部术语、你的代码库和设计文档使用的术语定义，以及如何找到这些定义。嗯，所以定制化说到底关于知识的定制化——它是在弥合「模型天然知道的」和「你的团队知道的」之间的差距。

[20:46] Um the fancy term for this if you want to sound fancy is called incontext learning which is a really fancy term that's uh basically means text files. Um, so the model weights uh are frozen at the point at which we release the model, right? You're not going to be changing those. And especially recently with the windown of fine-tuning interfaces and fine-tuning APIs, there's just less and less fine-tuning out there. Almost all of the customization that you're going to do happens in text space, which is actually really cool if you think about it, right? Because like programmers know how to control text, right? And so this gives you a lot of control over how you can uh improve the model without having to know anything about how LLMs work in terms of like weights, right? You don't need to know any of the math or anything like that. You know how to control text.

嗯，想说得高级点，这有个花哨的说法叫「上下文学习」（in-context learning），说白了基本就是文本文件。嗯，模型的权重在我们发布时就冻结了，对吧？你改不了那些权重。尤其是最近微调接口和微调 API 都在收缩，做微调的人越来越少。你几乎全部的定制工作都发生在文本空间里——仔细想想，这其实挺酷的，对吧？因为程序员懂怎么控制文本，对吧？所以你完全不用了解大语言模型权重的运作原理，就能对怎么改进模型有很大的掌控。你不需要懂任何数学之类，你懂怎么控制文本。

第 5 章 工具调用的现状与反馈闭环

The State of Tooling & Feedback Loops 工具仍很原始（ED 类比）；用 post-tool-use hook 构建「红色波浪线」式即时反馈。

[21:40] You can put text into the model and you get different output, right? Um it's not deterministic, but you can improve on the output of the model based on what you put into it. Um yeah. uh tooling. So, we already talked about what tool calls look like, but I think many of you will have noticed that this is like incredibly primitive, right? 你把文本输入模型，就会得到不一样的输出，对吧？嗯，它不是确定性的，但你确实可以靠输入的内容来改进模型的输出。嗯，对。嗯，工具这块。我们前面已经讲过工具调用长什么样，但我想你们很多人已经注意到，它原始得惊人，对吧？

[22:06] It's like literally Claude is roughly using Who here has used ED the Unix utility ED to like edit anything before? Who here knows what it is? Okay. Um, everyone I assume knows VI, right? If if the the the the bottom line of VI, if that was the entire editor, that's ED. So if you had to do the entirety of your editing by like find and replace, that that is roughly where agentic coding. This is roughly where agentic tooling is uh today, right? And no one has really figured out what does an IDE for quad look like? What does real time feedback for the model look like, right?

基本上，Claude 现在用的差不多就是……在座谁用过 Unix 工具 ED 来编辑东西？谁知道它是什么？好。嗯，我想大家都懂 VI 吧？如果把 VI 最底层的那一行当成整个编辑器，那就是 ED。所以如果你做所有编辑都得靠查找替换，那大致就是今天智能体编程、智能体工具所处的阶段，对吧？还没人真正搞明白：面向大模型的 IDE 长什么样？模型的实时反馈又长什么样，对吧？

[22:57] The gap between ED or VI like you know raw VI and VS code is the opportunity here and that's that's all happening in text space which is like very exciting, right? We're not like feeding things back into the weights in order to get the model to stop making mistakes while it's programming. we are actually just feeding text back into the model. So the the example I like to give is like what what do red squiggies look like for agents, right? How does an agent know that it's done something wrong in a tool call? Well, how do humans know that they've edited something wrong? Right?

ED 或原始 VI 和 VS Code 之间的差距，就是这里的机会；而这一切都发生在文本空间里，这非常令人兴奋，对吧？我们并不是把东西回灌进权重，来让模型在编程时停止犯错，我们只是把文本反馈回模型。所以我喜欢举的例子是：智能体的「红色波浪线」（red squiggles）长什么样？对吧？智能体

怎么知道自己在工具调用里做错了？那人类又是怎么知道编辑出错了的？对吧？

[23:34] If you if this variable doesn't exist um or isn't valid to reference in this context, right? You get a red squiggly, but obviously models get raw text. they don't get rich text like this, right? Um, and they don't get feedback as they type. So, we have to add that. Um, the mechanism we have for that in cloud code is a called a post tool use hook, which is when claude does the tool tool call that we showed earlier. As part of the the tool response block, we will tack on some information that you give us, right? And that information can look like this. But this is already something you know how to produce, right? Like your codebase already knows how to produce this because uh you used to produce it in order to like mouse over this red squiggly, right? That text exists, right? It's it's a nudge right at the moment of m the mistake, not when the model goes back and compiles later and has to think about, oh, what did I actually mean there? Using up more tokens, right?

如果这个变量不存在，或者在这个上下文里不能被引用，对吧？你会看到红色波浪线；但模型拿到的显然是原始文本，不是这种富文本，对吧？嗯，而且它们边打字也没有反馈。所以我们得自己加。嗯，Claude Code 里对应的机制叫 post-tool-use 钩子（工具使用后钩子）：Claude 发起前面那种工具调用时，作为工具响应块的一部分，我们会附上你提供的一些信息，对吧？这些信息可以长这样。但这其实是你已经能产出的东西，对吧？你的代码库本来就知道怎么产出它——你以前为了让鼠标悬停显示红色波浪线，就得产出它，对吧？那段文本是存在的，对吧？它是在犯错那一刻给的一个轻推（nudge），而不是等模型回头编译时才发现「哦，我刚才到底想写什么」，那样会消耗更多 token，对吧？

[24:42] Um, and there's lots of things you can put here, right? Type checking, uh, linting, uh, you can flag things that you would you were trying to get it to do in your cloud. MD, but it it didn't do. We're going to talk about how the mechanisms that we provide for for this later. But really, the fastest way to make an agent better at your code base isn't necessarily a smarter model. It's a tighter feedback loop, right? it's telling it that it's made a mistake sooner rather than later. Most of these are scripts you already have.

嗯，你可以在这里放很多东西，对吧？类型检查、代码 lint、你本来想在 CLAUDE.md 里让模型做、它却没做的事，都可以标出来。我们稍后会讲提供的这些机制。但说真的，让智能体在你的代码库上表现更好的最快方式，不一定是更聪明的模型，而是一个更紧密的反馈闭环，对吧？就是更早、而不是更晚地告诉它「你犯错了」。而这些大多是你已有的脚本。

[25:16] Um, another perspective on this is that uh for those of you who've worked in developer tooling before or developer experience before, there really are two kinds of tools. There's ones that kind of compensate for a lack of intelligence and there's ones that scale up with with intelligence. This is for humans or for models or for whatever, right? You can give a junior engineer something that stops them from editing certain files or stops them from changing certain things or running certain things and at some point they will become senior enough that you trust them to do something right and you have to give them a new set of tools for that. But if you can give if you can have a tool that gives a nudge or that gives a reminder that scales across because you know senior engineers still make these mistakes from time to time but they also know when it's not a mistake right so these like you want to focus on being well being AGI pilled sorry loaded buzzword but um it really is focusing on building the second kind of tools for your agents right thinking about not just what's going to happen, what's going to help the model now, but also what's going to help the model two generations from now when it is better

嗯，另一个角度看：做过开发者工具或开发者体验的人都知道，工具其实有两种。一种是在弥补智能的不足，另一种则能随着智能提升一起扩展。这对人类、对模型、对任何对象都成立，对吧？你可以给初级工程师一个工具，阻止他编辑某些文件、改动某些东西或运行某些命令；等他够资深、你信任他能做对时，你就得给他另一套工具。但如果你能有一个工具，给出提示或提醒，并且可以跨阶段扩展——因为资深工程师偶尔也会犯这些错，但他们知道什么时候其实没错——所以你要聚焦的是……抱歉又蹦出流行词，但真的就是要为你的智能体构建第二种工具：不只想着现在能帮到模型什么，也要想着两代模型之后、当它更强时还能帮到它什么。

第 6 章 上下文窗口：一个有限的盒子

The Context Window: A Finite Box 上下文窗口未随能力同步扩张；KV 缓存带来的扩展约束与「零开销」原则。

[26:34] or stronger in some way. Um, okay. The context window is a box. The context window is this space that you have to do this customization, right? The context window is where you can put your text.

……或在某方面更强时。嗯，好。上下文窗口（context window）就是一个盒子。上下文窗口是你用来做定制的那块空间，对吧？你的文本就放在上下文窗口里。

[26:51] Um over the past year the capabilities of models have kind of exploded. We went from like autocomplete fancy autocomplete to like kind of longtime horizon autonomous work. But like the context window sizes have not changed. Who's comfortable with the idea of a context window?

嗯，过去一年模型的能力可以说爆发了，我们从花哨的自动补全，走到了能长时间自主工作的阶段。但上下文窗口的大小却几乎没变。谁对「上下文窗口」这个概念比较熟悉？

[27:15] Okay, that's I think that's most people. um I will go through it in more detail but it is just like the set of tokens that a model can see in order to predict the next token right and that's limited fixed sized thing um and context windows really haven't changed that much like the first 1 million token context windows were like late 2024 um and the state-of-the-art Frontier models were 1 million token models in February of 2025 five. And right now the state-of-the-art Frontier models are roughly 1 million token context windows, right? That hasn't scaled up nearly as quickly as the capabilities of the model. Um, this means that as the tasks grow, you have to get smarter and smarter and smarter with what you actually put into this context window.

好，我想大部分人吧。嗯，我会更详细地讲，但它本质上就是模型为了预测下一个 token 所能看到的那些 token 的集合，对吧？它是一个有限的、固定大小的东西。上下文窗口其实变化不大——首个 100 万 token 的上下文窗口出现在 2024 年底，而最前沿的模型到 2025 年 2 月才到 100 万 token。现在最前沿的模型大致还是 100 万 token 的上下文窗口，对吧？它的扩展速度远没有模型能力那么快。嗯，这意味着随着任务变大，你往这个上下文窗口里放什么，必须越来越聪明、越来越讲究。

[28:08] Um, your entire like we figured out very early on that you can't just throw the entire codebase in there, right? But like documentation and um uh yeah internal codebase and reminders and all of these other things you have to cram this all into the context window. So um it's becoming a full-time engineering discipline. And I think there's definitely an aversion to the title context engineer that I see around a lot from uh software engineers.

嗯，我们很早就发现，你不能把整个代码库一股脑丢进去，对吧？但文档、内部代码库、提醒事项，以及所有这些东西，你又得全塞进上下文窗口。所以，嗯，它正在变成一门全职的工程学科。而且我确实看到很多软件工程师对「上下文工程师」（context engineer）这个头衔有些抵触。

[28:43] I kind of want to assert that these are the same thing, right? As uh agents get better and better at writing software, doing real software engineering, uh teaching them how to do it is going to become the main discipline of software engineering. Um and figuring out how to get good at that. Secretly, it's always been the main discipline of software engineering. It's just just that you had to get to a senior enough level where your full-time job was making sure that more junior engineers can do their jobs. Now, it's just that they're all agents, right? And it you have to be able to do it at every single level.

我倒想断言这两者其实是同一件事，对吧？随着智能体越来越擅长写软件、做真正的软件工程，「教它们怎么做」将成为软件工程的主要学科。嗯，而琢磨怎么把它做好，本来就是软件工程一直以来的主要学科——只不过以前你得熬到足够资深的级别，全职工作就是确保更初级的工程师能把活干好。现在只不过对象都变成了智能体，对吧？而且你得在每一个层级上都能做到这一点。

[29:21] Um so, yeah, the context window is where we have to put everything that contributes to the prediction of a of the next token, right? Everything that um Claude can think about in a given token prediction or when it's creating a tool call when it's making an edit has to fit into this space. So you have like your system prompt, your tool definitions, plotted MD skills, any files you've read, any tool results, all of that. And the more stuff you put up here up front, the less room you have to do actual tasks um further back. So every customization competes with

the space to do work and your whole codebase doesn't fit. Um and and it you can't just kind of naively dump everything into the box, right? You can, but like that's not going to perform nearly as well as very carefully selecting the relevant context.

嗯，对，上下文窗口就是我们必须放入「一切有助于预测下一个 token」的东西的地方，对吧？

Claude 在某一轮 token 预测时所能想到的、或在创建工具调用、做编辑时所能依据的一切，都必须装进这块空间。所以你有系统提示词、工具定义、CLAUDE.md、技能（skill）、你读过的任何文件、任何工具结果，全都在里面。你 upfront 塞进去的东西越多，后面留给实际任务的空间就越少。所以每一项定制都在和工作空间竞争，而你的整个代码库是装不下的。嗯，你不能天真地把所有东西都倒进盒子，对吧？你可以这么做，但效果远不如精心挑选相关上下文来得好。

[30:16] Um the analogy I like to use for this is that it is like running npm on an Arduino, right? When you are running a package manager on your computer, you can do all kinds of things to like deal with traditional um management problems, right? If you have diamond diagram or sorry, diamond problems, diamond dependency problems in in PM, you can literally just import two different versions of the library to resolve it. You can't do that when you're customizing a context, right? you have to decide what is the more relevant piece of information intelligently and put that into the context and leave out the things that aren't relevant. Um, so the the customization really does have to be don't pay for what what you don't use. Zero overhead principle um or you're going to run out of tokens very very quickly.

嗯，我喜欢用的类比是：这就像在 Arduino 上跑 npm，对吧？你在电脑上跑包管理器时，可以用各种手段处理传统的依赖管理问题，对吧？如果你有菱形依赖（diamond dependency）问题，你大可以直接引入库的两个不同版本来解决。但在定制上下文时你做不到，对吧？你必须聪明地判断哪条信息更相关，把它放进上下文，而把不相关的排除掉。嗯，所以定制确实必须遵循「不为没用到的东西付费」——零开销原则，否则你的 token 会非常非常快地耗尽。

[31:12] There's one more constraint that I actually really want to go over um because I don't see this in a lot of talks about this kind of thing and that is the KV cache. Um the way that next token prediction works in modern LLMs.

还有一个约束，我特别想讲一下，因为很多同类演讲都不提它，那就是 KV 缓存（KV cache）。嗯，现代大语言模型里「下一个 token 预测」的运作方式就是这样。

[31:29] Um this will probably change at some point but I it's been about to change for a year and a half now or two years now and I still haven't seen a lot of progress on the problem space. um it's a lot lot cheaper if all of these uh tokens are the same as the previous time you try to predict these tokens, right?

嗯，这一点将来某天大概会变，但它「即将改变」已经说了一年半甚至两年，我还没看到这个问题空间有多少进展。嗯，如果这次预测的这些 token 和上一次完全一样，成本就会低非常多，对吧？

[31:53] So like in computing in general when you're looking at the shape of problem we've been talking about so far, right? Where you have relevant information and less relevant information and a really confined space to to put all of that into. You do something that looks like an LRU cache, right? This is how CPU caches work. This is how uh web caches work. Right? We we we kind of this space has been explored. But the problem is that because of this constraint where like um all of the things before your token have to be the same in order to predict the next token.

所以一般来说，面对我们前面讨论的那种问题形态——有相关信息、有不太相关的信息，还有一个非常受限的存放空间——你通常会想到类似 LRU 缓存（最近最少使用缓存）的方案，对吧？CPU 缓存是这么干的，Web 缓存也是这么干的，对吧？这个领域我们其实已经探索过了。但问题在于，由于那个约束——为了预测下一个 token，你之前的所有 token 都必须和上一次保持一致。

[32:33] um like this prediction costs 10 times more than this prediction and if you do that you know every tool call you will end up with a massive massive token spin. So very early on um if I understand correctly at least cursor used to do this with cursor rules. They would kind of swap them in and out based on what was the most relevant rule for the particular thing that it was working on and then it would evict an old rule that wasn't related and and

very quickly they found out that like that costs a whole lot. Um, and so this problem is more difficult and more nuanced than just doing an LRU cache of the relevant information and calling it a day. Um, so that that's the other like constraint here that's like difficult to explain without going deep on that. But this the caching of the prediction mechanism enables this to be very very cheap whereas this whole thing is very expensive.

嗯，像这样一次预测的成本是另一次预测的 10 倍；如果你在每个工具调用里都这么做，最终会陷入巨大的 token 自旋（token spin）。所以很早以前——如果我理解没错的话——Cursor 曾经用 cursor rules 这么干过：他们根据当前任务最相关的规则，把规则换进换出，再把不相关的旧规则驱逐出去，结果很快就发现这代价非常高。嗯，所以这个问题比「对相关信息做个 LRU 缓存就完事」要更难、更微妙。嗯，这就是另一个约束，不深入讲很难说清。但重点是：预测机制的缓存让「这个」变得非常便宜，而「那一整套」则非常昂贵。

第 7 章 插件原语 (Plugin Primitives)

Plugin Primitives 企业级定制机制：MCP、Skills、Sub-agents、Hooks，以及为何 CLAUDE.md 不是好选择。

[33:38] Um all right so I'm going to dive into the so plugins are the mechanism that we provide for customization especially at enterprise scale um of cloud code and I want to talk through each of these uh the the plug-in abstractions that you can build plugins out of and talk about like what happens when you throw this into a monor repo with a 100 million lines of code right where or if you're not a monor repo company what happens when you have 5,000 different repos that all have information that you need to know in order to do something in one or two of those repos.

嗯，好，接下来我深入讲讲插件。插件是我们在 Claude Code 里、尤其是企业级规模上提供的定制机制。我想逐个讲讲你能用来构建插件的那些原语（abstraction），并讨论当你把它丢进一个有 1 亿行代码的单体仓库时会怎样；或者你要是不是单体仓库公司，当你有 5000 个不同的仓库、而其中只有一两个需要用到某些信息时，又会怎样。

[34:16] Um, and so the question there, right, is does it scale, right? You if you have 5,000 different repos and they all want to provide three or four different skills to explain how to work with that their own part of the repo, right? Um, what happens? what happens if each one of them provides their own MCP server.

嗯，所以这里的问题是：它能扩展吗？对吧？如果你有 5000 个不同的仓库，每个都想提供三四个技能（skill）来说明怎么用它的那部分代码，对吧？嗯，会发生什么？如果每个仓库都提供自己的 MCP 服务器，又会怎样？

[34:37] Um, so we're going to walk through that. We're going to walk through that in the context of four specific plug-in primitives. There are other plug-in primitives, but a lot of the logic kind of transfers there. Uh, anyway, we'll dive in. Um, who here knows what MCP is?

嗯，我们就顺着这个思路来讲。我们会围绕四种具体的插件原语展开。还有其他插件原语，但很多逻辑是相通的。嗯，总之我们开始吧。嗯，在座谁知道 MCP 是什么？

[34:53] Okay, I think most people have heard of it, but I don't think most people have thought about it in terms of customizing uh agentic coding uh environments. So if you are going to build some integration with the um with anthropic in general, you probably should send out a public MCP server, right? Or you should put out an MCP server for that integration. But this is about how do you customize your codebase to interact with other developers at your company. Uh MCP is this protocol that exposes it. It basically just adds more schemas for tool calls to your system prompt, right?

好，我想大多数人都听过，但我估计大多数人没从「定制智能体编程环境」的角度想过它。所以你要和我们 Anthropic 做某种集成，你大概应该对外发布一个公开的 MCP 服务器，对吧？或者至少为那个集成提供一个 MCP 服务器。但这里讲的是：你如何定制自己的代码库，让它和公司里其他开发者互动。嗯，MCP 就是暴露这种能力的协议，它本质上只是往你的系统提示词里多加一些工具调用的 schema，对吧？

[35:35] It gives a way of doing that. It's a JSON based protocol. Um it gives a way like a very lightweight from the client side way of the um client to say here I want extra tools to deal with this thing, right? Um it is transport agnostic. It has a lot of nice properties for consumers that are not necessarily great for software engineers. They can be, but um pretty rarely. So when is it the right tool?

它提供了一种实现方式，是一个基于 JSON 的协议。嗯，它让客户端能用一种非常轻量化的方式说「我想加些额外工具来处理这个东西」，对吧？嗯，它是传输层无关的（transport agnostic）。它对使用者有很多不错的属性，但对软件工程师未必友好——倒也不是绝对，只是很少见。所以，什么时候它才是合适的工具？

[36:06] Yeah, if you need to design this for any client, it needs to work with a chatbot. Um the server side owns the authentication, which is like a really big portability win, but then you have to actually set all of this up to work in any environment. And the fact is that like you're not programming in any environment. you are programming in the development environment of your company, right? Um, if you already have a command line interface for something, it's probably a lot better to create a skill that explains how to use the CLI for for your uh product. So, does it scale? This is like what we're looking at. What happens if you have a thousand MCP servers in your monor repo and you try and just load them all into context?

嗯，当你需要为任意客户端设计、它得能配合聊天机器人工作时，它就合适。嗯，服务端掌握认证（authentication），这在可移植性上是很大的胜利；但代价是你得真的把这一整套在任何环境下都搭起来。而事实是，你并不是在「任意环境」里编程，你是在自己公司的开发环境里编程，对吧？嗯，如果你已经有某个命令行接口（CLI），那更好的做法通常是创建一个技能（skill），说明怎么用这个 CLI 来操作你的产品。那么，它能扩展吗？这正是我们要看的：如果你的单体仓库里有一千个 MCP 服务器，你试着把它们全都加载进上下文，会发生什么？

[36:51] Well, every tool has a name, a description, and a schema that sits in the system prompt so that the model knows how to use it. If you have 20 MCP servers and they each have 15 tools, that ends up with a context window that's mostly full of um tool descriptions and you have very little room to work, right?

嗯，每个工具都有名字、描述和 schema，它们都放在系统提示词里，好让模型知道怎么用。如果你有 20 个 MCP 服务器，每个有 15 个工具，那你的上下文窗口基本就被工具描述填满了，留给工作的空间所剩无几，对吧？

[37:12] Um so, no, it doesn't scale without help. What we have been working on recently is a thing called tool search which loads the names of all the tools up front. Uh so it doesn't still like completely scale because if you have I don't know literally a million tools then you're going to use your entire context but it scales a little better than um name plus description plus schema. So, we we give the names and then we give Claude a tool to search for tools, which yeah, um, turtles all the way down. But, um, the problem is that that like there's not a lot of free lunch here. Um, the more descriptive name you give or especially if you include like some set of some subset of the description, the more likely Claude is to remember to search for the tool.

嗯，所以，不加辅助的话，它确实扩展不了。我们最近在做的一个东西叫「工具搜索」（tool search），它会先加载所有工具的名字。嗯，这当然也不是完全能扩展——你要是真有一百万个工具，照样会把上下文用完——但它比「名字+描述+schema」全塞进去要稍微好扩展一点。所以我们只给名字，再给 Claude 一个「搜索工具」的工具——对，套娃到底（turtles all the way down）。但嗯，这里没有太多免费午餐。嗯，你给的名字越具描述性，尤其是你要是还包含一部分描述，Claude 就越可能在需要时用搜索找到这个工具。

[38:07] Um and if you don't give much if you have a like random tool named query then uh cloud will very rarely search for it at the right time. So you do still have to think about how many MCP servers am I going to give if you use cloud code you should be thinking about how many MCP servers am I giving it access to because that is going to dilute the amount uh dilute its ability to find the tools it needs to do its job. So, it it does scale up a little bit with tool search. Um, but not great. Uh, who knows what skills are.

嗯，你要是不给足够的信息——比如有个随便叫「query」的工具——那 Claude 基本不会在正确的时候去搜索它。所以你还是得想清楚：我要给它多少个 MCP 服务器？你要用 Claude Code，就该想清楚「我到底给了它多少 MCP 服务器的访问权」，因为这会稀释它找到完成工作所需工具的能力。所以，有了工具搜索，它确实能稍微扩展一点。嗯，但谈不上多好。嗯，谁知道什么是技能（skill）？

[38:45] Okay, cool. Um, skills are just folders, folders with a markdown file in them. Um, the basic idea here is that a skill is a um kind of a lazy system prompt, right? It has a bunch of instructions and it has some sort of summary of those instructions that the model can use to decide when it wants to expand this part of the system prompt.

好，酷。嗯，技能（skill）其实就是文件夹——里面带一个 markdown 文件的文件夹。嗯，基本思路是：技能是一种「惰性系统提示词」（lazy system prompt），对吧？它有一堆指令，还有对这些指令的某种摘要，模型可以据此判断什么时候该展开系统提示词的这一部分。

[39:08] Right? This is how a lot of kind of scaling you have a like a a small thing and then you can turn it into a big thing on demand. This is how a lot of scaling in software engineering has always worked, right? Um and uh yeah, so you can you can also have um other resources in the folder with the skill and cloud will know how to get to them. Um but the uh I think the thing that has resonated with a lot of people about this is the fact that it has such a simple form factor, right? It's just a folder. There's no special protocol.

对吧？很多扩展都是这样：先有个小东西，需要时再把它展开成大东西。软件工程里很多扩展一直都是这么运作的，对吧？嗯，对，你也可以在技能的文件夹里放其他资源，Claude 知道怎么去取用。嗯，但我觉得很多人对它产生共鸣的点，正是它极其简单的形式，对吧？就只是一个文件夹，没有特殊协议。

[39:46] There is like front matter fields to the skill.md file, but there are things like description and then you put this like short description that tells the model whether or not it wants to expand the um the full prompt. Give me a second. I'm going to take a drink of water. It is warm up here.

在 SKILL.md 文件里是有 front matter 字段的，比如 description——你写上一段简短描述，告诉模型要不要展开完整的提示词。稍等一下，我喝口水，上面有点热。

[40:07] Are people following more or less where I'm going? Okay, I'm seeing a lot of nods. So, I'm taking that as a somewhat good sign. I guess the people that I have lost are asleep so that's okay too. Um it's it's a lot right like this is a software engineering is a field that is had that's been around for 60 70 years and we're having more change in the past two years than the rest of it right in in some ways at least uh or at least arguably u we can talk about whether or not we have but all right um so skills so we so skills are these lazy system prompts right it has a description of when Claude wants to use the thing, it expands to a much larger amount of text when Claude decides that it is relevant. Um, so does it scale? Right. The the whole body is pay-per-use. So we are getting like don't pay for the body if you don't use it. But um and so like you know at a small codebase scale you know maybe 500,000 lines I would call 500,000 lines of code a small codebase. Um, and I think many of you probably work in enterprises where you would call that a small enterprise codebase, right? Um,

大家大体还跟得上我的思路吗？好，我看到很多人点头，那我就当这是个还算好的信号。我想那些跟丢的人应该都睡着了，那也没关系。嗯，这信息量很大，对吧？软件工程这门学科存在了六七十年，而我们过去两年的变化，在某些方面比之前所有年份加起来都多——至少可以说有争议，但姑且这么说。嗯，好，所以技能（skill）就是这些惰性系统提示词，对吧？它有一段描述说明 Claude 什么时候该用它；当 Claude 判断相关时，会展开成一大段文本。嗯，那它能扩展吗？对吧。整个正文是「按需付费」的，所以我们做到了「不用就不为它付费」。但在小规模代码库——比如 50 万行，我会把 50 万行称为小代码库——嗯，我想你们很多人在企业里也会把那叫小型企业代码库，对吧？嗯，

[41:24] the the body is paid per use. The description is always loaded, right? The description always goes in the system prompt. So, if you have a 100,000 skills and you load a 100,000 descriptions into the system prompt,

you are going to use your entire context window, right? So it it still doesn't completely scale up to a large enterprise like Google's codebase or Facebook's codebase. Um and I think for me at least that's the interesting question about how do we get these things to do software engineering?

正文是按次付费的，但描述（description）总是会被加载，对吧？描述永远在系统提示词里。所以你要是有 10 万个技能（skill），把 10 万条描述都加载进系统提示词，你的整个上下文窗口就全满了，对吧？所以它仍然无法完全扩展到像 Google 或 Facebook 代码库那样的大型企业规模。嗯，而至少对我而言，这才是「怎么让这些东西真正做软件工程」这个有趣问题的关键。

[41:58] They need to be able to scale up in that way. Um skills also currently don't have a way to do hierarchy. We're working on that uh hopefully out soon. So it it does scale uh kind of it's kind of the same problem that we talked about in terms of uh tool search right tool search has this um like description this this description line that if you make it longer and longer it it like triggers better um yeah sub agents what's up we working on that So the question was, do we need a skill search tool? We're working on that. It's um it's a little harder um because it requires the model to know what it doesn't know versus tool search, the model is much more likely to know what it can't do.

它们必须能那样扩展。嗯，技能（skill）目前也还没有层级结构（hierarchy），我们正在做，希望很快能出。所以它在扩展性上……有点像我们前面讲的工具搜索的问题：工具搜索也有那条描述行，你把它写得越长，触发效果越好。嗯，对，子智能体（sub-agent）——怎么了，我们也在做。所以当时的问题是：我们需要一个技能搜索工具吗？我们在做。嗯，这稍微难一点，因为它要求模型知道自己「不知道什么」，而工具搜索里模型更容易知道自己「做不到什么」。

[42:55] Does that make sense? Like it's a lot easier to tell that you can't do something than to tell that you don't know something. Um, but it, yeah, that is a direction we're going. Sub aents are relatively similar to skills. I'm going to go through this pretty quickly because I actually want to get to the nice stuff at the end. Um, where you you basically give it a short description that gets put into the system prompt, but Claude instead of Claude expanding that text into its own context window, it'll expand that text into a separate context window and then get back a summary of what the sub agent did, right? it'll expand that text along with like its own prompt into a a sub aent context window, a separate context window and then gets back a summary that that is that is compact. Right? So it's in this sense it actually scales better but across a um monor repo a large monor repo where you have hundreds of these or thousands of these or tens of thousands of these you're going to start using up large fractions of your context window on these description strings. Right? So um but the yeah the takeaway for understanding the difference here is that skills are in context sub aents are

这说得通吗？判断「你做不到某事」比判断「你不知道某事」容易得多。嗯，但没错，这正是我们的一个方向。子智能体（sub-agent）和技能（skill）比较相似。我会讲得快一点，因为我想把后面精彩的部分留出来。嗯，你基本上给它一段简短描述放进系统提示词；但和技能不同，Claude 不是把这段文本展开进自己的上下文窗口，而是把它展开进一个独立的上下文窗口，再拿回一段关于子智能体做了什么的摘要，对吧？它会把这段文本连同自己的提示一起，展开到一个独立的子智能体上下文窗口里，然后拿回一段紧凑的摘要。对吧？所以从这个意义上说，它的扩展性其实更好；但在一个有几百、几千甚至几万个这种东西的大型单体仓库里，你还是会开始把上下文窗口的很大一部分耗在这些描述字符串上。对吧？所以，嗯，理解二者区别的关键结论是：技能在上下文内，子智能体在——

[44:05] out of context. I think we think that the the difference is fundamental enough uh to keep them as separately named things but there's a lot of similarity there. I think we're still just we're still figuring out how we want to name these things, how we want to separate these things, what the um ontology of all of this is. I don't think anyone really knows what the long-term state of this is, but hooks are my favorite abstraction to talk about in this context because hooks do scale. Uh, spoiler, right? So, this is this is like the kinds of abstractions that I think we generally want to be building are the abstractions that don't put anything at all into the context window uh unless you somehow determine it is relevant.

……上下文外。我认为这个区别足够根本，所以还是把它们分开命名；但二者确实有很多相似之处。

我觉得我们也还在摸索：该怎么命名、该怎么区分、整个本体论（ontology）是什么。我不认为有人

真正知道长期会是什么样。不过钩子（hook）是我在这种情况下最爱讲的抽象，因为钩子确实能扩展。嗯，剧透一下，对吧？所以，我认为我们总体上应该去构建的那类抽象，是「除非你判定它相关，否则完全不往上下文窗口里放任何东西」的抽象。

[44:51] Right? So hooks are an abstraction that we have uh in cloud code that will when a certain event happens like a tool use or a user submits a prompt or the model stops um working for one reason or another um or the the whole context compacts. You can run a specific script and there's like there is a protocol for like what the input to that script looks like and what output you have to deliver and based on how you deliver that output we'll just we will put um some context into the window into the context window for you or not. Um, but I mean the the way to think about how this scales, right, is if you have if you are working on a Rust project and you have 10 uh JavaScript skills, you still pay for those 10 short prompts related to JavaScript even though you're not doing anything related at all. And you have to remember to manually toggle them off to not use that. Right? If you have 10 skills for linting your JavaScript code and you are working on a Rust codebase, you don't pay well, you pay for like running the script that immediately sees you're not writing JavaScript and then goes away, right? You don't pay for anything you don't use except for like

对吧？所以钩子是 Claude Code 里的一种抽象：当某个事件发生时——比如一次工具调用、用户提交了提示词、模型因某种原因停下、或者整个上下文被压缩——你可以运行一个特定的脚本。它有一套协议，规定这个脚本的输入长什么样、你必须返回什么输出；而根据你返回输出的方式，我们会（或不会）往你的上下文窗口里放一些上下文。嗯，但要理解它怎么扩展，可以这样想：如果你在做 Rust 项目，却有 10 个 JavaScript 的技能（skill），你仍然要为那 10 段和 JavaScript 相关的简短提示付费，尽管你根本没在碰 JavaScript，而且你得记得手动把它们关掉。对吧？但如果你有 10 个用于 lint JavaScript 代码的技能，而你在写 Rust 代码库，你其实——你只是为「运行那个脚本、它立刻发现你没在写 JavaScript 然后就退出」付了点费，对吧？你不为任何没用到的东西付费，除了

[46:11] CPU resources, which I think we can all agree are a little bit less limited than context space. Um, yeah. So, it runs outside the context window. You can do them, um, you can do them as scripts. You can also actually put an agent in here if you really want to. Um, probably use that sparingly because you'll use a whole lot of tokens really quickly. Um, does it scale? Uh, well, we already talked about this. Uh, it in only injects something when it fires, when it matches. the the pattern that we have is like call the script, determine if it's relevant immediately, and quick exit if not, and otherwise do some processing to figure out if there's anything relevant that you can tell the model. Um, this is where the red squiggles live as we talked about earlier.

……CPU 资源，这点我想我们都同意：它比上下文空间要宽裕得多。嗯，对。所以它在上下文窗口之外运行。你可以用脚本实现，嗯，如果你真想，甚至可以在里面放一个智能体。嗯，但最好省着点用，因为那样 token 会消耗得飞快。嗯，它能扩展吗？嗯，这个我们已经讲过了：只有当它触发、匹配时，它才注入东西。我们的模式是：调用脚本，立刻判断相不相关，不相关就快速退出；否则再做些处理，看看有没有什么相关的东西可以告诉模型。嗯，前面说的红色波浪线，就住在这里。

[46:59] Um, what are some abstractions that I don't consider to be good parts of plugins? This is like a pretty quick aside, so I'm going to go through it, but the one that I really like to talk about here is cloud.MD.

嗯，有哪些抽象我认为不算插件的好组成部分？这是个很快的题外话，我简单过一下，但我特别想讲的是 CLAUDE.md。

[47:11] I have had so many people ask me why do plugins not have a claw.md abstraction and I think hopefully if you followed the talk up to this point you'll pretty you should understand this right it it is always it is the worst at uh don't pay for what you don't use right because if it if we were to provide an abstraction that allowed plugins to unconditionally inject a large amount of text at the very beginning of every context then we'd limited to five or 10 plugins being up being uh active at any given time.

有太多人问我：为什么插件没有一个 CLAUDE.md 抽象？我想如果你一直跟到了这里，应该已经能理解了，对吧？它恰恰是「不为没用到的东西付费」这一原则的反面典型，因为如果我们要提供一个抽

象，允许插件在每个上下文的最开头无条件注入一大段文本，那我们同时能激活的插件就只能限制在五到十个。

[47:48] And and the biggest problem with that is that like it doesn't look like it's that expensive, right? If you're writing out your plugin, probably the first thing you're going to do is create a cloud. MD file that explains what the plugin is.

而最大的问题是：它看起来并不那么昂贵，对吧？当你在写一个插件时，大概率第一件事就是建一个 CLAUDE.md 文件，说明这个插件是干什么的。

[47:58] Like it's so low cost on the implementation side and it's so high cost um on the uh sorry yeah it is so high cost on the use side, right? So it it you can still do this, right? You can still set up a session start hook that injects a whole bunch of tokens into your context right at the beginning of your of every single session. But at least that makes it obvious that you're doing something for that cost everyone no matter what.

它在实现侧成本低得可怜，而在使用侧的成本却高得离谱，对吧？所以你当然还是可以这么做，对吧？你完全可以设一个「会话开始」钩子，在每个会话的最开头往上下文里注入一大堆 token。但至少那样会让代价变得显眼——无论什么情况，你都在为它付出成本。

第 8 章 记忆 (Memory) 与边界

Memory & Its Boundary 记忆由模型管理，应与上下文工程原语区分；过渡到第二部分。

[48:29] Um memory. Memory is another one that I like to say kind of establishes a different boundary of what I would call a plugin, right? Memory is like this model curated. Who here is familiar with model memory or agentic memory? A few people uh spoiler alert, it's text files. Um all of its text files is just you just tell the model to write a text file and then like read the text file at a very specific point later, right? But it is because it's model curated. I don't consider it to be a context engineering primitive. Right? We really want to focus on what are the primitives for sustainable reusable context engineering. And I think I'd like to keep that separate from what we call a plug-in. So even if they are shaped like skills, even if they are things that are that are updated and contain the same kinds of knowledge of as skills, I really think that the future of software engineering at at ultra-large scale is going to have to be very discriminating about what we call context engineering and what we call memory. Those two things are need going to need to function very separately. So, I throw that in because I think I think I've I hope at this point in the talk I've

嗯，记忆 (memory)。记忆是另一个——我想说它划定了另一种「插件」的边界，对吧？记忆是模型来管理的。谁熟悉模型记忆或智能体记忆？有几个人，剧透一下，它就是文本文件。嗯，全是文本文件：你只是告诉模型去写一个文本文件，然后在之后某个非常特定的时刻再去读它，对吧？但正因为它是由模型管理的，我不把它看作上下文工程的原语。对吧？我们真正想聚焦的是：什么是可持续、可复用的上下文工程的原语？而我想把那个「插件」分开。所以即便它们长得像技能 (skill)，即便它们会被更新、包含和技能同类别的知识，我依然认为，超大规模软件工程的未来必须非常严格地区分我们所谓的「上下文工程」和「记忆」——这两件事必须非常独立地运作。所以我把这点抛出来，因为我希望讲到这里时，我已经——

[49:46] given you the tools to understand why I believe that. Um, all right. The the part where we talk about some of the things that the cloud code team uses these days and where I see this going in 2026.

……给了你们理解我为何这么认为的工具。嗯，好。接下来这部分，我们聊聊 Claude Code 团队如今在用的一些做法，以及我对 2026 年走向的看法。

第 9 章 规模化与并行：工作树、身份与团队

Scaling Up: Worktrees, Identity & Teams 用 worktree / 持久身份 / agent teams / slashloop / auto mode 实现多智能体并行。

[49:59] Um, the only way to scale up from here, the only way to speed up from here is to do multiple things at the same time. Um, how many people would say that your regular workflow uses more than Well, let's say how many people would say your regular workflow uses more than one clot at a time, more than one session? How about more than five, more than 20?

嗯，要在此基础上继续扩展、继续提速，唯一的办法就是同时做多件事。嗯，有多少人会说自己的日常工作流一次用不止一个 Claude (clot)？不止一个会话？那超过五个呢？超过二十个呢？

[50:26] Um, okay. So, some somewhere around there. But you can pretty quickly say see how if you could get to the point where you're comfortable managing 20 clouds at a time, you could get close to four times as much work done if as if you can manage five. So I think a lot of the kind of work in 2026 is going to be figuring out abstractions that give developers the uh cognitive space they need to context switch between these things quickly. Um get work trees are the simplest way to get started on this. put each session on a different work tree so that you're not the clouds aren't stepping on each other. I don't know that I need to go too far on that, but one of the things that really helps with this when you have multiple sessions, who has used slashreame or slashcolor, it's okay. It's uh three people should be everyone. If you've used everyone who raised their hand about multiple clots, this this like works wonders. It's super super simple, super super easy. Just label what it's working on. Um give it a color. when you switch to the window that's doing it, your m your brain will see the color. It'll trigger some memory of what's going on there. Um, there's

嗯，好，大概就在那个量级附近。但你很快就能看出：如果你能从容地同时管理 20 个 Claude，相比只能管理 5 个，你完成的工足量能接近 4 倍。所以我认为 2026 年很多工作，会是去设计那些给开发者提供「认知空间」的抽象，让他们能快速在这些东西之间切换上下文。嗯，git worktree 是入门最简单的方式：让每个会话待在不同的 worktree 上，这样 Claude 们就不会互相踩脚。这个我不多展开了，但有一点对多会话真的很有帮助——谁用过 /rename 或 /color？没关系，就三个人，应该人人都用过。所有刚才举手说用多个 Claude 的人，这招效果惊人，超级简单、超级容易：就给它在做什么贴个标签，嗯，再给它上个颜色。当你切到那个窗口时，你的大脑看到颜色，就会唤起对那块工作的记忆。嗯，有——

[51:40] some research to show that humans like recall things related to color or non-colorblind humans uh recall things related to color faster than they recall things related to pros. Um, my actual setup looks like this. I actually have these longunning agents.

……一些研究表明，人类——或者说非色盲人类——对与颜色相关事物的回忆，比对文字相关事物的回忆更快。嗯，我实际的配置大概是这样的：我真的养着一些长时运行的智能体 (long-running agent)。

[51:58] Um, this is a little outdated because I um didn't feel like I could operate efficiently with just 16 permanent work trees. So now this goes all the way to Z instead of A through F. But each one of them is separate. They're kind of long running and they're set up. They're they're parked in one spot and there's an agent that knows that its responsibility is to maintain that work tree long term. So um and for me at least having an agent with a persistent name and a persistent memory um it kind of manages its own like scratch files within its own uh branch of the work tree. Um allows me at least as someone who is has experience being a tech lead or a of a large organization where you have a bunch of different people coming to you over the course of the day telling you what they've done. having persistent identities really helps me.

嗯，这图有点过时了，因为我发现光有 16 个常驻 worktree 不够高效，所以现在是从 A 到 Z，而不是 A 到 F。但它们每一个都是独立的，算是长时运行、已配置好的。它们停在各自的位置上，每个都有一个智能体，清楚自己的职责是长期维护那个 worktree。所以，嗯，至少对我来说，一个拥有持久名字和持久记忆的智能体，会在它自己 worktree 的分支里管理自己的临时文件。嗯，这让我——一个有过技术负责人经验、白天有一堆不同的人来跟你汇报他们做了什么的人——觉得「持久身份」真的很有帮助。

[52:53] Um, so I think it's worth worth trying. This is like diagrammatically what it looks like. I have like long lived work trees that are that have names corresponding to the agent name. Uh, and they all track upstream main and they know that they're supposed to manage that tracking of upstream name.

嗯，所以我觉得值得一试。这就是它的示意图：我有长生命周期的 worktree，名字和智能体名字对应；嗯，它们都跟踪上游的 main，并且知道自己应该负责维护对上游的跟踪。

[53:12] So they don't step on each other even from a um, so it's like I mean it's like they're separate developers, right? Um anyway, uh yeah, I have a whole talk that I'd like to develop out about like how does agent identity work with cognitive load? Um so I don't know. I think that's a very interesting thing to explore over the course of 2026.

所以它们不会互相踩脚，即便从——嗯，我的意思是，它们就像彼此独立的开发者，对吧？嗯，总之，我还有一整场演讲想展开讲：智能体身份是如何与认知负荷相互作用的？嗯，所以说不准，但我觉得这是 2026 年一个非常值得探索的方向。

[53:35] Um agent teams. Has anyone here tried agent teams before? Oh, that breaks my heart. We need more. Um, so agent teams are ways the the basic thing here is just giving the model ways to talk to other agents. So if you're running two clouds at the same time and you tell one of them one thing, you can tell it to tell the other one the same thing if it needs it, right? Um, send message tool is the the baseline primitive that gives Claude the ability to send one send a message from one session to another.

嗯，agent teams。在座有人试过 agent teams 吗？哦，这让我有点伤心，我们需要更多人用。嗯，agent teams 的核心，就是给模型一种和其他智能体交谈的方式。所以如果你同时跑两个 Claude，你告诉其中一个某件事，它可以（在需要时）告诉另一个同样的事，对吧？嗯，「发送消息」（send message）工具是最基础的原子能力，它让 Claude 能从一个会话向另一个会话发消息。

[54:09] Um, you can have a a team leader that delegates to teammates. You can also just have uh a bunch of agents that are all peers of each other. And if one of them discovers something, you can tell it to share it with the others or you can tell it not to. This is all under your control. We are moving towards I actually don't know if it's been launched because I've been out of office for a couple of weeks but it is announced enough um that we're moving towards a direction where if you are signed in in a certain way and you give it permission then any session on any machine that you have can talk to any session on any other machine that you have um hopefully um so that you just have the ability to tell the model to talk to other instances. Uh, and this is like another example of like give Claude access to all of the things that you have access to, right? Because it needs to be able to see the things you can see. And if another agent is one of the things you can see, then you should be giving it access to that, right? Um, slash oh, babysitting um, slashloop is something we shipped maybe two months

嗯，你可以有一个团队负责人（team leader）向下属委派任务；也可以就放一堆彼此平等的智能体。如果其中一个发现了什么，你可以让它分享给其他成员，或者不分享——这都由你掌控。我们正在朝这个方向走：我其实不确定它是否已经发布，因为我休了两周假；但已经宣布得够多了——方向是：如果你以某种方式登录并授予权限，那么你任意机器上的任意会话，都能和你其他机器上的任意会话对话，嗯，希望如此——这样你就有了让模型去和其他实例交谈的能力。嗯，这又是「给 Claude 访问你能访问的一切」的一个例子，对吧？因为它需要能看到你能看到的东西；如果另一个智能体也是你能看到的东西之一，那你就该给它这个访问权，对吧？嗯，/oh 有个叫 babysitting 的，嗯，slashloop 是大概两个月前——

[55:15] ago. Um, it is uh literally a cron command or a cron tool for the model that it can schedule a message to just show up every 10 minutes. It has the ability to turn itself off at some point. So if it's like, okay, that's enough. I've seen this every 10 minutes for 3 days now. I'm just going to turn it off. um uses some some amount of tokens obviously, but um it surprisingly minimal too compared to human having to go and check CI and then copy and paste and get the thing started up again. I find often that the problem is that the model goes to sleep before it's finished the job. And this really helps with that. It also I mean I think it also gives Claude the tools to make sure it doesn't do that, right? Right. And we see that increasingly the models are better at understanding

the fact that like if something is going to come in as a result in 3 hours that it should set something to wake itself up in 3 hours.

……发布的。嗯，它 literal 就是给模型的一个 cron 命令或 cron 工具，能让模型安排一条消息每 10 分钟冒出来一次。它还有能力在某个时刻自己关掉——比如「好了，够了，这玩意每 10 分钟来一次已经连着三天了，我把它关掉」。嗯，它显然会消耗一些 token，但相比人类得自己去查 CI、复制粘贴、再把流程重新启动，它的消耗出奇地小。我经常发现的问题是：模型常常在活儿干完之前就「睡着」了，而这招真的能治这个。而且，嗯，我认为这也给了 Claude 确保自己不睡着的工具，对吧？对。我们看到，模型越来越擅长理解这一点：如果某结果要 3 小时后才出来，它就该设个 3 小时后把自己唤醒的东西。

[56:15] Um auto mode. Who's using auto mode? Should be everyone hopefully. Um yeah, this is really what enables this multi- clotting, right? This is how you get up to 20 or 30 agents, right? So you're not just sitting there hitting inner all day. Um it is better than YOLO mode. We have a sophisticated set of uh safeguards that go through several screenings of of models and classifiers that uh check whether something is dangerous before blocking it or allowing it. It looks through all of your prompts and sees if you've actually told it to do something or not.

嗯，auto mode。谁在用 auto mode？希望是所有人。嗯，对，这才是实现「多 Claude 并行」的真正关键，对吧？这就是你能拉起 20 甚至 30 个智能体的方式，对吧？这样你就不用整天坐在那儿不停按回车。嗯，它比 YOLO 模式更好：我们有一套复杂的 safeguards，会经过好几轮模型和分类器的筛查，在阻止或放行前先判断某件事是否危险。它会审阅你所有的提示词，看你是否真的吩咐它去做那件事。

第 10 章 2026 展望与收尾

2026 Outlook & Closing 2025 是「信息进模型」，2026 是「信息出模型」；fleet view 与 remote control，三点重申。

[56:53] And if you haven't told it to do something that seems dangerous, it'll stop it from doing it. Um, this is what makes sloop and agent teams and overnight runs actually work.

如果你没让它做那件看似危险的事，它就会拦住它。嗯，正是这个，才让 slashloop、agent teams 和通宵运行真正跑得起来。

[57:03] It's a it's a little expensive. It it costs uh anywhere from 10 to 40% more depending on the model. Um, I am running out of time and I'm sorry, but um, this is really what I'm getting at here and I think I've said this a few times, right? 2025 was about getting information into the model to get better results because it Agent intake programming was in its infancy and we were constantly compensating for the model making mistakes. 2026 I believe is going to be a lot more about getting information from the model to the user.

它稍微贵一点，根据模型不同，成本大概高出 10% 到 40%。嗯，我时间不够了，抱歉，但这就是我想说的核心，而且我想我已经说过几次了，对吧？2025 年是关于「把信息送进模型」以获得更好结果，因为智能体编程还在起步期，我们一直在为模型的错误打补丁。而我相信 2026 年会更多是关于「把信息从模型传递给你」。

[57:40] Right? The quality of the information that the user gives the model is going to matter less and less as the models get stronger and it's going to be less and less of a bottleneck. Um, and this year it's going to be a lot more about how quickly can you context switch, how quickly can you scale up your workflows to get information about what the model is doing. Uh, Claude put it this way. Um, your attention is the smallest box in the system, which I think is cute. Um, so cloud agents is actually one of the things we've launched along those lines if you haven't tried it or it's also called fleet view. Has anyone tried this yet?

对吧？随着模型变强，用户提供给模型的信息质量，重要性会越来越低，也越来越不再是瓶颈。嗯，而今年会更关乎：你能多快切换上下文，多快扩展你的工作流，以获取「模型正在做什么」的信息。

嗯，Claude 是这么说的：你的注意力是系统里最小的那只盒子——我觉得这话挺可爱。嗯，所以 cloud agents（也就是 fleet view（集群总览））就是我们沿这个方向推出的东西之一，如果你还没试过的话。有人试过吗？

[58:20] One person. Okay, everyone should try this. It is It is amazing. Um the team member who put this together literally hit I think a thousand PRs one week using this um or yeah it is um significant way to scale up your attention, right? Because it is showing everything that's running all in one place. We have little classifier models that give a very short description of like what the model needs, what it has just done. Um, and these are like, you know, 10 different cloud code sessions all in one view. Um, and yeah, the lightweight classifier and you can jump in between sessions from directly in there. No more like 20 different tabs.

一个人。好，大家都该试试，它真的惊人。嗯，搭出这个的团队成员，靠它一周 literal 合了上千个 PR。嗯，对，它是扩展你注意力的重要方式，对吧？因为它把所有正在运行的东西都集中在一个地方展示。我们有小型的分类模型，会给出极简短的描述：模型现在需要什么、刚刚做了什么。嗯，就是比如 10 个不同的 Claude Code 会话，全在一个视图里。嗯，对，那个轻量分类器让你能直接在里面跳转到不同会话之间，再也不用开 20 个不同的标签页了。

[59:08] Uh, remote control, another fantastic thing that I use all the time. Um, I have these kind of persistent agents running on a server in the cloud and I can uh interact with them on my phone. I interact with them on cloud code desktop. Um, I wrote portions of this talk literally while walking over from my hotel by interacting with the agent on my phone that was responsible or sorry on my dev box that was responsible for this talk. The three things I do want to reiterate before I run out of time, I have 30 seconds. um give the model access. Think about the box and think about what you're putting into the box and how that affects what you're getting out. Uh and pick abstractions that scale up to large scale real world software real world software engineering. Um thanks for hanging out hanging with me. I'm sorry that my uh stumbling over my words was a little worse than usual. It is also after five for me. But I hope you all enjoyed it.

嗯，remote control，另一个我天天在用的好东西。嗯，我有一些常驻的智能体跑在云端的服务器上，我可以用手机和它们交互，也可以在 Claude Code 桌面端和它们交互。嗯，我这场演讲的某些部分，就是我在从酒店走过来的路上，通过手机和那个负责这场演讲的智能体——抱歉，是在我的开发机上、负责这场演讲的那个——交互着写出来的。时间快到了，我还有 30 秒，有三点想重申：嗯，给模型访问权；想清楚那个盒子，想清楚你往盒子里放了什么、又如何影响了你得到的东西；嗯，选择能扩展到大规模真实软件工程的抽象。嗯，感谢你们陪我、听我讲。抱歉我今天说话有点磕绊，比平时还厉害——对我来说也已经过了五点了。但我希望你们都 enjoy 了这场。

[60:09] I'll be around. Uh you can always hit me up on Twitter. Um if you have questions about this, I'm happy to answer them. Um yeah, thanks for your time and thanks for like hanging out at the end of the day.

我会在这附近。嗯，你们随时可以在 Twitter 上找我。嗯，如果对此有疑问，我很乐意解答。嗯，对，谢谢你们的时间，也谢谢你们在一天结束时还留下来。

[60:24] >> [applause]

>> [掌声]